

Task 01

- ① The code iterates through the array using nested loops. For each element $arr[i]$ it checks if there exists another element $arr[j]$ such that:
 $arr[i] + arr[j] = target$. if found it returns the pair of indices else it returns "impossible" $\rightarrow O(n^2)$
- ② Initializes two pointers i and t to the beginning and the end of the array. ~~while i is~~ runs a while loop until $i == t$ and checks if $arr[i] + arr[t] == target$. if true returns the ~~indices~~ indices. if sum is greater than target t moves to smaller elements. if sum is less than t increments i to move to the larger elements. if a pair is not found "impossible" is returned. $\rightarrow O(n)$

Task 2 :

① $n \log n$ solution :

This solution merges the 2 arrays and sorts them using python's builtin function `.sort()` which is a heap sort method

② $O(n)$ solution :

At first we create an empty list of the length of the two arrays combined. then we initialize pointers $k \rightarrow$ to keep track of current index of the empty list and i, j to keep track of current indexes of array 1 and array 2 respectively. Then it runs a while loop simultaneously on both arrays until the length of any of the arrays is reached. Inside the loop we check if `arr1[i] < arr2[j]` and we append the smaller value into the empty list and we increment the pointer which points to the appended element. we also increment k . If any of the lengths of the arrays is reached the elements in the ~~arrays~~ ^{other array} are appended in the next indices in order as there is nothing to compare.

Task 3:

- Sorts the tasks according to end times and then start taking tasks if the ~~start~~ end time of the current task and the next task does not overlap the task is taken this is known as the greedy approach

Task 4:

- Sorts the task according to end times and then assigns the task to the ones who have to wait the least time if the current task of that person and the next task does not overlap. and a counter is set to count the number of tasks assigned.